

Práctica de Repaso (MD)

Pasaje de mensajes

1. En una oficina existen 100 empleados que envían documentos para imprimir en 5 impresoras compartidas. Los pedidos de impresión son procesados por orden de llegada y se asignan a la primera impresora que se encuentre libre:

a) Implemente un programa que permita resolver el problema anterior usando PMA

```
chan EMPL_IMPR(documento);

PROCESS Empleado[idE: 1..100]
{
    documento doc;
    while (true) {
        // Enviar documento
        doc = cargarDocumento();
        send EMPL_IMPR(doc);
    }
}

PROCESS Impresora[idI: 1..5]
{
    documento doc;
    while (true) {
        // Recibir documento
        send IMPR_COORD(id);
        receive COORD_IMPR(doc);
        // Realizar impresión
        imprimir(doc);
    }
}
```

```

    }
}

```

- A menos que se pida algún control o prioridad sobre lo que se recibe del empleado, no hace falta utilizar un buffer.

[Corregido]

b) Resuelva el mismo problema anterior pero ahora usando PMS

```

PROCESS Empleado[]
{
    documento doc;
    while (true) {
        doc = cargarDocumento();
        Coordinador!envio(doc);
    }
}

// Este proceso introduce orden de llegada de por sí
PROCESS Coordinador
{
    cola cola_documentos;
    documento doc;

    while (true) {
        do Empleado[*]?envio(doc) → push(cola_documentos, doc);
        □ not empty(cola_documentos); Impresora?pedido() → pop(cola_d
ocumentos, doc);
                                                                    Impresor
a!pedido(doc);

        od
    }
}

PROCESS Impresora[idl: 1..5]

```

```

{
    documento doc;
    while (true) {
        Coordinador!pedido();
        Coordinador?pedido(doc);
        imprimir(doc);
        // ¿Me comunico con el empleado? -- NO HACE FALTA, PREGUNTAR
    }
}

```

- ¿Hace falta el while cuando solo tengo un do no determinístico? Una ayudante me había dicho que si por el caso en el que no se activara ninguna guarda, pero en la teoría dice que, si no tiene condición, esta se asume True y la guarda se bloquea, no falla. → [Consultar](#)

[Corregido]

2. Resolver el siguiente problema con PMS. En la estación de trenes hay una terminal de SUBE que debe ser usada por P personas de acuerdo con el orden de llegada. Cuando la persona accede a la terminal, la usa y luego se retira para dejar al siguiente. Nota: cada Persona usa sólo una vez la terminal

```

PROCESS Persona[id: 1..P]
{
    BufferTerminal!aviso(id);
    Terminal?permitir_uso();
    // Usar terminal
    Terminal!termino_uso();
}

PROCESS BufferTerminal
{
    idp: int;
    cola_ids: cola;
}

```

```

while (true) do
  do Persona[*]?aviso(id) → cola_ids.push(id);
  □ not empty(cola_ids); Terminal?aviso() → pop(cola_ids, idp);
  Terminal!mandar_id(idp);
end;
}

PROCESS Terminal
{
  idp: int;
  for i:= 1 to P do
    BufferTerminal!aviso();
    BuferTerminal?mandar_id(idp);
    Persona[idp]!permitir_uso();
    Persona[idp]?termino_uso();
  end;
}

```

[Corregido]

3. Resolver el siguiente problema con PMA. En un negocio de cobros digitales hay P personas que deben pasar por la única caja de cobros para realizar el pago de sus boletas. Las personas son atendidas de acuerdo con el orden de llegada, teniendo prioridad aquellos que deben pagar menos de 5 boletas de los que pagan más. Adicionalmente, las personas embarazadas tienen prioridad sobre los dos casos anteriores. Las personas entregan sus boletas al cajero y el dinero de pago; el cajero les devuelve el vuelto y los recibos de pago

```

chan AVISO(int);
chan ENTREGA_EMB(int, int, int);
chan ENTREGA_CINCO(int, int, int);
chan ENTREGA_NORMAL(int, int, int);
chan RESPUESTA[N](int, recibo);

```

PROCESS Persona[idP: 1..P]

```
{
    int cant_boletas = ...;
    bool esta_embarazada = ...;
    int vuelto;
    recibo rec;

    if (esta_embarazada)
        send ENTREGA_EMB(cant_boletas, vuelto, idP);
    else if (cant_boletas < 5)
        send ENTREGA_CINCO(cant_boletas, vuelto, idP);
    else
        send ENTREGA_NORMAL(cant_boletas, vuelto, idP);

    send AVISO(1);
    receive RESPUESTA(vuelto, rec);
}
```

PROCESS Cajero

```
{
    int ok;
    int cant_boletas, dinero, idP, vuelto;
    bool esta_embarazada;
    recibo rec;

    while (true) do
        recibir AVISO(ok);
        if not empty(ENTREGA_EMB(cant_boletas, dinero, idP)) →
            receive ENTREGA_EMB(cant_boletas, dinero, idP);
        □ not empty(ENTREGA_CINCO(cant_boletas, dinero, idP)) && empty
(ENTREGA_EMB) →
            receive ENTREGA_CINCO(cant_boletas, dinero, idP);
        □ not empty(ENTREGA_NORMAL(cant_boletas, dinero, idP)) && empty
ty(ENTREGA_CINCO) && empty(ENTREGA_EMB) →
            receive ENTREGA_NORMAL(cant_boletas, dinero, idP));
        rec, vuelto = generarRecibo(cant_boletas, dinero);
    end while
}
```

```
end;  
  
send RESPUESTA[idP](vuelta, rec);  
}
```

[Corregido]

ADA

1. La página web del Banco Central exhibe las diferentes cotizaciones del dólar oficial de 20 bancos del país, tanto para la compra como para la venta. Existe una tarea programada que se ocupa de actualizar la página en forma periódica y para ello consulta la cotización de cada uno de los 20 bancos. Cada banco dispone de una API, cuya única función es procesar las solicitudes de aplicaciones externas. La tarea programada consulta de a una API por vez, esperando a lo sumo 5 segundos por su respuesta. Si pasado ese tiempo no respondió, entonces se mostrará vacía la información de ese banco.

```
procedure Ej1ADA is  
  
    // Declaraciones  
  
    Task Actualizador is  
    End Actualizador;  
  
    Task Type Banco is  
        Entry pedirCotizacion(cot: OUT cotizacion)  
    End Banco;  
  
    arrBancos: (1..20) of Banco;  
  
    // Implementaciones
```

```

Task Body Actualizador is
    cot: cotizacion
Begin
    for i:= 1 to 20 loop
        // Consultar la API del i-ésimo banco
        SELECT
            arrBancos[i].pedirCotizacion(cot)
        DELAY 5
        cot:= "Vacio";
    END SELECT;

    // Actualizar información de la página para el banco
    // Asumir que existe
    actualizarPagina(i, cot);
end loop;
End Actualizador;

```

```

Task Body Banco is
    cot: cotizacion;
Begin
    ACCEPT pedirCotizacion(cot: OUT cotizacion) DO
        cot:= cargarCotizacion();
    END pedirCotizacon;
End Banco;

```

```

Begin
    null;
End Ej1ADA;

```

[Corregido]

2. Resolver el siguiente problema. En un negocio de cobros digitales hay P personas que deben pasar por la única caja de cobros para realizar el pago de sus boletas. Las personas son atendidas de acuerdo con el orden de llegada, teniendo prioridad aquellos que deben pagar menos de 5 boletas de los que pagan

más. Adicionalmente, las personas ancianas tienen prioridad sobre los dos casos anteriores. Las personas entregan sus boletas al cajero y el dinero de pago; el cajero les devuelve el vuelto y los recibos de pago

```

procedure Ej2ADA is

    // Declaraciones

    Task Cajero is
        Entry EntregarAnciano(cant_boletas: IN int; dinero: IN int; id: IN int)
        Entry EntregarBoletaMayor(cant_boletas: IN int; dinero: IN int; id: IN int)
    nt)
        Entry EntregarNormal(cant_boletas: IN int; dinero: IN int; id: IN int)
        Entry RecibirId(id: OUT int);    End Cajero;
    End Cajero;

    Task Type Persona is
        Entry Ident(id: IN int);
        Entry RecibirVueltoPago(dine: IN int; rec: OUT recibo);
    End Persona;

    // Implementaciones

    arrPersonas: array (1..P) of Persona;

    Task Body Cajero is
        id: int;
        rec: recibo;
        vuelto: int;
    Begin
        while (true) loop
            SELECT
                ACCEPT EntregarAnciano(cant_boletas: IN int; dinero: IN int; id: IN int) DO
                    vuelto:= calcularVuelto(dinero);

```



```

        rec:= generarRecibo(cant_boletas);
        id:= id;
    END EntregarAnciano;
OR
    WHEN (EntregarAnciano'count == 0) ⇒ ACCEPT EntregarBoletaMayor(cant_boletas: IN int; dinero: IN int; id: IN int) DO
        vuelto:= calcularVuelto(dinero);
        rec:= generarRecibo(cant_boletas);
        id:= id;
    END EntregarBoletaMayor;
OR
    WHEN (EntregarAnciano'count == 0 && EntregarBoletaMayor'count == 0) ⇒ ACCEPT EntregarNormal(cant_boletas: IN int; dinero: IN int; id: IN int) DO
        vuelto:= calcularVuelto(dinero);
        rec:= generarRecibo(cant_boletas);
        id:= id;
    END EntregarNormal;
END SELECT;

    Persona[id].RecibirVueltoPago(vuelto, rec);
end loop;
End AdminCajero;

Task Body Persona is
    cant_boletas: int;
    dinero: int;
    edad: int;
    id: int;
    recib: recibo;
Begin
    cant_boletas:= ...;
    dinero:= ...;
    edad:= ...;

    // Recibir ID

```

```

ACCEPT Ident(id: IN int) DO
    id:= id;
END Ident;

// Controlar para las prioridades
// Asumo que la función esMayor(int) existe
if (esMayor(edad))
    Cajero.EntregarAnciano(cant_boletas, dinero, id);
else if (cant_boletas < 5)
    Cajero.EntregarBoletaMayor(cant_boletas, dinero, id);
else
    Cajero.EntregarNormal(cant_boletas, dinero, id);

ACCEPT RecibirVueltoPago(dine: IN int; rec: OUT recibo) DO
    dinero:= dinero + dine;
    recib:= recib;
END RecibirVueltoPago;

End Persona;

Begin
    // Se hace necesario otorgar identificadores para así
    // el Cajero puede comunicarse con cada Persona
    for i:= 1 to P loop
        arrPersonas.Ident(i);
    end loop;
End Ej2ADA;

```

- Si hay mas de un elemento que tenemos que priorizar, está mal utilizar un tipo de dato especial, y en su lugar tenemos que aprovechar las Entry Calls que ofrece ADA.

[Corregido]

3. Resolver el siguiente problema. La oficina central de una empresa de venta de indumentaria debe calcular cuántas veces fue vendido cada uno de los artículos de su catálogo. La empresa se compone de 100 sucursales y cada una de ellas maneja su

propia base de datos de ventas. La oficina central cuenta con una herramienta que funciona de la siguiente manera: ante la consulta realizada para un artículo determinado, la herramienta envía el identificador del artículo a las sucursales, para que cada una calcule cuántas veces fue vendido en ella. Al final del procesamiento, la herramienta debe conocer cuántas veces fue vendido en total, considerando todas las sucursales. Cuando ha terminado de procesar un artículo comienza con el siguiente (suponga que la herramienta tiene una función generarArtículo() que retorna el siguiente ID a consultar). Nota: maximizar la concurrencia. Existe una función ObtenerVentas(ID) que retorna la cantidad de veces que fue vendido el artículo con identificador ID en la base de la sucursal que la llama.

```
// Enviar el id del artículo a las sucursales
// Cada una calcula cuantas veces fue vendido dentro

procedure Ej3ADA is

    // Declaraciones

    Task Herramienta is
        Entry ObtenerIdProducto(id_producto: OUT int);
        Entry ObtenerVentas(cant_ventas: IN int);
    End Herramienta;

    Task Type Sucursal
        Entry Avanzar();
    End Sucursal;

    // Implementaciones

    arrSucursales: arr (1..100) of Sucursal;

    Task Body Herramienta is
        id_producto: int;
```

```

    veces: int;
    cant_total: int;
Begin
    // Enviar el ID del artículo a las 100 sucursales
    while (true) loop
        id_producto := generarArticulo();
        cant_total:= 0;
        veces:= 200;

        while (veces > 0) loop
            SELECT
                ACCEPT ObtenerIdProducto(id_producto: OUT int)
                    id_producto:= id_producto;
            END ObtenerIdProducto;
            veces:= veces - 1;
        OR
            ACCEPT DevolverCantidadVentas(cant_ventas: IN int)
                cant_total:= cant_total + cant_ventas;
            END DevolverCantidadVentas;
            veces:= veces - 1;
        END SELECT;
    end loop;

    // AVISARLE A LAS SUCURSALES QUE PUEDEN AVANZAR
    for i:= 1 to 100 loop
        arrSucursales[i].Avanzar();
    end loop;
end loop;
End Herramienta;

Task Body Sucursal is
    id_producto: int;
    cant_ventas: int;
Begin
    while (true) loop
        // ¿Estaría bien invertir la comunicación?

```

```
Herramienta.ObtenerIdProducto(id_producto);

// Calcular cuantas veces fue vendido dentro
cant_ventas:= ObtenerVentas(id_producto);

Herramienta.DevolverCantidadVentas(cant_ventas);

// AGREGAR AVISO SIGUIENTE PARA VANZAR
ACCEPT Avanzar();
end loop;
End Sucursal;

Begin
End Ej3ADA;
```

[Corregido]